

FULL-STACK EXPENSE TRACKER

React + Spring Boot + Firebase Firestore CRUD Web Application

Build a complete, professional, responsive **Expense Tracker Web Application** that satisfies a college **CRUD-Based Web Application Development SOP**.

The application must demonstrate:

- Create
 - Read
 - Update
 - Delete
 - REST API
 - Firebase Firestore database
 - React frontend
 - Spring Boot backend
 - Validation
 - Search
 - Filtering
 - Error handling
 - Dashboard
 - Expense calculations
 - Testing with Postman
 - GitHub-ready structure
 - Complete documentation
 - Responsive design
-

1. PROJECT NAME

Use:

Expense Tracker

Repository name:

expense-tracker

2. TECHNOLOGY STACK

Frontend

Use:

- React
- JavaScript
- Vite
- HTML5
- CSS3
- React Router
- Axios or Fetch API
- Lucide React icons

Backend

Use:

- Java
- Spring Boot
- Spring Web
- Spring Validation
- Firebase Admin SDK

Database

Use:

- Firebase Firestore

Do NOT use MySQL, PostgreSQL, SQLite, or MongoDB.

Testing

Use:

- Postman

Version Control

Use:

- Git
- GitHub

3. ARCHITECTURE

Use this architecture:

User ↓ React Frontend ↓ HTTP / JSON ↓ Spring Boot REST API ↓ Service Layer ↓ Repository Layer ↓
Firebase Admin SDK ↓ Firebase Firestore

The React frontend should communicate with the backend API.

Do not make the main expense CRUD functionality depend on localStorage.

Expenses must be permanently stored in Firebase Firestore.

4. CORE FUNCTIONALITY

The application must allow the user to:

1. Add an expense
 2. View all expenses
 3. View one expense
 4. Edit an expense
 5. Delete an expense
 6. Search expenses
 7. Filter expenses
 8. Filter by category
 9. Filter by payment method
 10. Filter by date
 11. Calculate total expenses
 12. Calculate monthly expenses
 13. Calculate category-wise expenses
 14. Display dashboard statistics
 15. Validate expense data
 16. Display success/error messages
-

5. EXPENSE DATA MODEL

Create an Expense model with:

- id
- title

- description
- amount
- category
- paymentMethod
- date
- createdAt
- updatedAt

Required:

- title
 - amount
 - category
 - paymentMethod
 - date
-

6. EXPENSE CATEGORIES

Provide predefined categories:

- Food
- Transport
- Shopping
- Education
- Bills
- Entertainment
- Health
- Travel
- Groceries
- Other

Allow the category list to be centrally maintained.

7. PAYMENT METHODS

Provide:

- Cash
- UPI
- Debit Card
- Credit Card
- Bank Transfer
- Other

8. FIRESTORE DATABASE

Create a Firestore collection:

```
expenses
```

Example document:

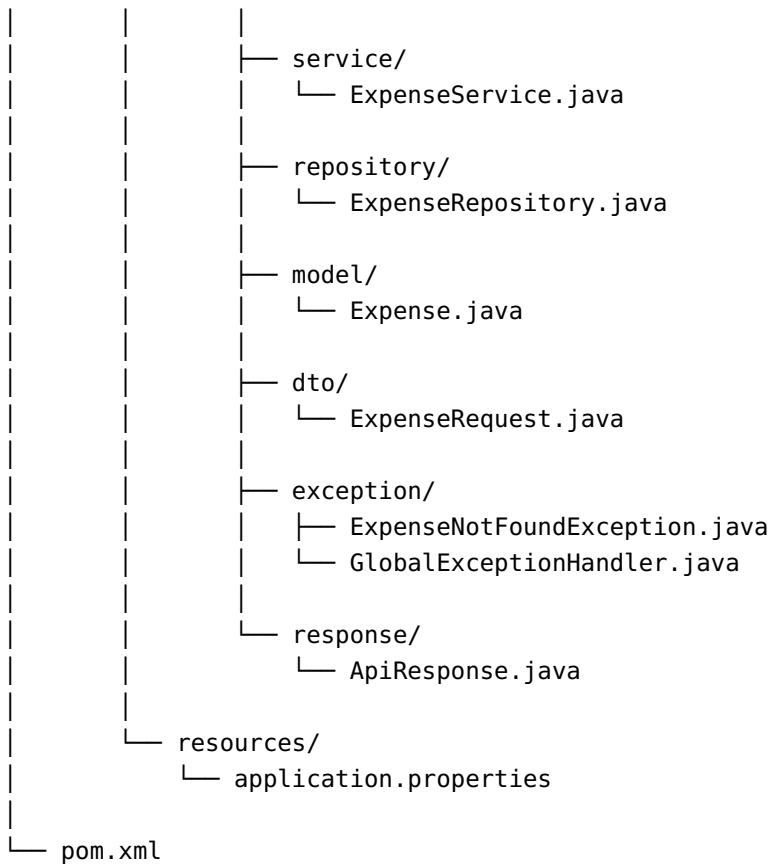
```
{
  "id": "unique-document-id",
  "title": "Lunch",
  "description": "College lunch",
  "amount": 120,
  "category": "Food",
  "paymentMethod": "UPI",
  "date": "2026-09-16",
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

Use Firestore timestamps for createdAt and updatedAt where appropriate.

9. BACKEND STRUCTURE

Create:

```
backend/
|
|— src/
|   |— main/
|       |— java/
|           |— com/example/expensetracker/
|               |— ExpenseTrackerApplication.java
|               |— config/
|                   |— FirebaseConfig.java
|                   |— CorsConfig.java
|               |— controller/
|                   |— ExpenseController.java
```



Use proper separation:

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Firebase Firestore
```

10. FIREBASE SECURITY

Never hard-code:

- Firebase private key
- service account credentials
- passwords
- secrets

Use environment variables or secure local configuration.

Add credentials and `.env` files to `.gitignore`.

Create:

```
.env.example
```

with placeholders only.

Never commit actual Firebase credentials to GitHub.

11. REST API

Implement:

CREATE EXPENSE

```
POST /api/expenses
```

Example request:

```
{
  "title": "Lunch",
  "description": "College lunch",
  "amount": 120,
  "category": "Food",
  "paymentMethod": "UPI",
  "date": "2026-09-16"
}
```

Return:

```
{
  "success": true,
  "message": "Expense created successfully",
  "data": {}
}
```

12. GET ALL EXPENSES

```
GET /api/expenses
```

Return all expenses.

13. GET ONE EXPENSE

```
GET /api/expenses/{id}
```

If the ID does not exist:

```
404 Not Found
```

Return a clean JSON error.

14. UPDATE EXPENSE

```
PUT /api/expenses/{id}
```

Allow the user to edit:

- title
- description
- amount
- category
- payment method
- date

Update the `updatedAt` timestamp.

15. DELETE EXPENSE

```
DELETE /api/expenses/{id}
```

Delete the expense from Firestore.

Do not only remove it from React state.

16. VALIDATION

Implement frontend and backend validation.

Title

Required.

Minimum reasonable length.

Amount

Required.

Must be greater than 0.

Do not allow:

```
0
negative values
invalid text
```

Category

Required.

Must be a valid category.

Payment Method

Required.

Must be a valid payment method.

Date

Required.

Do not allow invalid date values.

17. ERROR HANDLING

Handle:

- Invalid input
- Expense not found
- Firebase failure
- Network failure
- Backend unavailable
- Server error

Example:

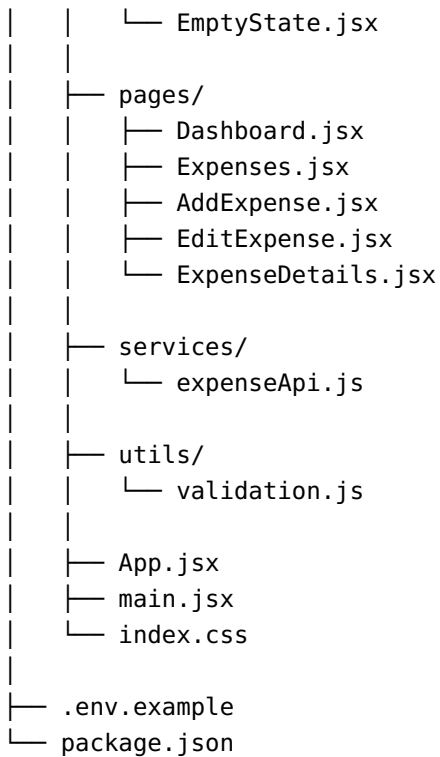
```
{
  "success": false,
  "message": "Expense not found",
  "error": "EXPENSE_NOT_FOUND"
}
```

Never expose Java stack traces to the frontend.

18. FRONTEND STRUCTURE

Create:

```
frontend/
|
├─ src/
|   └─ components/
|       ├── Navbar.jsx
|       ├── Sidebar.jsx
|       ├── ExpenseForm.jsx
|       ├── ExpenseTable.jsx
|       ├── ExpenseCard.jsx
|       ├── SearchBar.jsx
|       ├── FilterPanel.jsx
|       ├── StatCard.jsx
|       ├── DeleteModal.jsx
|       └── Loading.jsx
```



19. UI DESIGN

Create a modern finance dashboard.

Design requirements:

- Professional
- Clean
- Modern
- Responsive
- Easy to understand
- Mobile-friendly
- Good typography
- Consistent spacing
- Rounded cards
- Subtle shadows
- Clear icons
- Attractive dashboard

Avoid an overly complicated interface.

20. MAIN NAVIGATION

Create:

Dashboard
Expenses
Add Expense
About

Use React Router.

Highlight the current page.

Make navigation responsive.

21. DASHBOARD

Create a professional dashboard.

Display:

Total Expenses

Example:

₹12,450

This Month

₹4,250

Number of Expenses

38

Average Expense

₹327

These values must be calculated from actual Firestore/API data.

Do not hard-code them.

22. CATEGORY SUMMARY

Display category-wise spending.

Example:

Food	₹2,500
Transport	₹1,200
Shopping	₹3,000
Education	₹1,500
Bills	₹2,000

Calculate these dynamically.

23. EXPENSE LIST

Create a professional table.

Columns:

Date
Title
Category
Payment Method
Amount
Actions

Actions:

View
Edit
Delete

Example:

Date	Title	Category	Method	Amount
16 Sep	Lunch	Food	UPI	₹120
15 Sep	Bus	Transport	Cash	₹50
14 Sep	Book	Education	UPI	₹450

24. ADD EXPENSE PAGE

Create a form containing:

Title

Description

Amount

Category

Payment Method

Date

Buttons:

Cancel
Add Expense

After successful creation:

Expense added successfully

Then navigate back to the expense list.

25. EDIT EXPENSE

When the user clicks Edit:

1. Fetch the expense
 2. Populate the form
 3. Allow editing
 4. Validate
 5. Send PUT request
 6. Update Firestore
 7. Show success notification
 8. Return to the list
-

26. EXPENSE DETAILS

Create a details page/modal.

Display:

Expense Title

Description

Amount

Category

Payment Method

Date

Created At

Updated At

Buttons:

Edit

Delete

Back

27. DELETE CONFIRMATION

When Delete is clicked:

Show:

Delete Expense?

Are you sure you want to delete this expense?
This action cannot be undone.

[Cancel] [Delete]

After successful deletion:

Expense deleted successfully

Refresh the list.

28. SEARCH

Add search functionality.

Search by:

- title
- description
- category
- payment method

Example:

Search expenses...

Results should update dynamically.

29. FILTERING

Implement:

Category

All
Food
Transport
Shopping
Education
Bills
Entertainment
Health
Travel
Groceries
Other

Payment Method

All
Cash
UPI
Debit Card
Credit Card
Bank Transfer
Other

Date

Provide useful date filtering such as:

- All
- Today
- This Week
- This Month
- Custom Date Range

Allow search and filters to work together.

30. SORTING

Add optional sorting:

- Newest first
- Oldest first

- Amount: High → Low
 - Amount: Low → High
-

31. DASHBOARD CHARTS

Create useful visualizations using a suitable lightweight chart library.

Include:

Category Spending

Show spending by category.

Monthly Spending

Show monthly spending trend.

Charts must use actual API data.

Do not hard-code chart values.

32. LOADING STATES

When API calls are running:

Display:

Loading expenses...

or a spinner/skeleton.

Disable duplicate submissions while saving.

33. EMPTY STATE

If no expenses exist:

No expenses found.

Start tracking your spending by adding your first expense.

[Add Expense]

If filters return no results:

No expenses match your search or filters.

34. SUCCESS NOTIFICATIONS

Show toast notifications:

Expense added successfully

Expense updated successfully

Expense deleted successfully

35. API SERVICE

Create:

services/expenseApi.js

Functions:

```
createExpense()  
getExpenses()  
getExpenseById()  
updateExpense()  
deleteExpense()
```

Centralize API requests.

Do not duplicate API URLs across components.

Use:

```
VITE_API_BASE_URL
```

for the frontend API URL.

36. CORS

Configure Spring Boot CORS correctly for the React frontend.

Do not use an unnecessarily permissive production configuration.

Make the frontend URL configurable.

37. RESPONSIVE DESIGN

Support:

- Desktop
- Laptop
- Tablet
- Mobile

The application must remain usable on small screens.

On mobile:

- Sidebar should collapse
 - Tables can become cards or horizontally scroll
 - Forms should use one-column layouts
 - Dashboard cards should stack
-

38. ACCESSIBILITY

Use:

- Proper labels
 - Semantic HTML
 - Keyboard-accessible buttons
 - Good contrast
 - Clear error messages
 - Accessible modal controls
 - Meaningful button text
-

39. BACKEND EXCEPTION HANDLING

Create:

```
GlobalExceptionHandler.java
```

Handle:

- ExpenseNotFoundException
- Validation errors
- Firebase errors
- Illegal arguments
- General exceptions

Return consistent JSON responses.

40. HTTP STATUS CODES

Use appropriate status codes:

```
200 OK
201 Created
400 Bad Request
404 Not Found
500 Internal Server Error
```

Use:

409 Conflict

where appropriate.

41. POSTMAN TESTING

Create a Postman collection:

```
Expense Tracker API
|
├─ Create Expense
├─ Get All Expenses
├─ Get Expense By ID
├─ Update Expense
├─ Delete Expense
├─ Invalid Expense
├─ Invalid Expense ID
└─ Server Error Test
```

42. TEST CASES

Include:

ID	Test	Expected Result
TC01	Create valid expense	Expense created
TC02	Empty title	Validation error
TC03	Amount = 0	Validation error
TC04	Negative amount	Validation error
TC05	Invalid category	Validation error
TC06	Invalid payment method	Validation error
TC07	Get all expenses	Expenses returned
TC08	Get existing expense	Expense returned
TC09	Invalid expense ID	404
TC10	Update expense	Expense updated

ID	Test	Expected Result
TC11	Delete expense	Expense deleted
TC12	Search expense	Matching results
TC13	Category filter	Correct results
TC14	Date filter	Correct results
TC15	Backend unavailable	Error message

43. GITHUB STRUCTURE

Repository:

```
expense-tracker/  
|  
├─ frontend/  
├─ backend/  
├─ docs/  
├─ screenshots/  
├─ README.md  
└─ .gitignore
```

Do not commit:

```
node_modules/  
target/  
.env  
Firebase service-account JSON  
private keys  
passwords
```

44. GIT COMMITS

Use meaningful commits:

```
Initial project setup  
Create Spring Boot backend  
Configure Firebase
```

```
Create Expense model
Create Firestore repository
Implement service layer
Implement REST controller
Implement POST API
Implement GET API
Implement PUT API
Implement DELETE API
Add validation
Add exception handling
Create React frontend
Create dashboard
Create expense form
Create expense list
Implement API integration
Add search and filters
Add charts
Improve responsive design
Add documentation
Final project cleanup
```

45. README DOCUMENTATION

Create a complete README:

```
# Expense Tracker

## Project Overview

## Problem Statement

## Objectives

## Features

## Technology Stack

## System Architecture

## Project Structure

## Firebase Setup

## Firestore Database Structure
```

```
## Backend Setup

## Frontend Setup

## Environment Variables

## How to Run

## REST API Documentation

## CRUD Operations

## Validation

## Search and Filtering

## Dashboard

## Testing

## Screenshots

## Challenges and Solutions

## Future Enhancements

## GitHub Repository
```

46. PROJECT REPORT

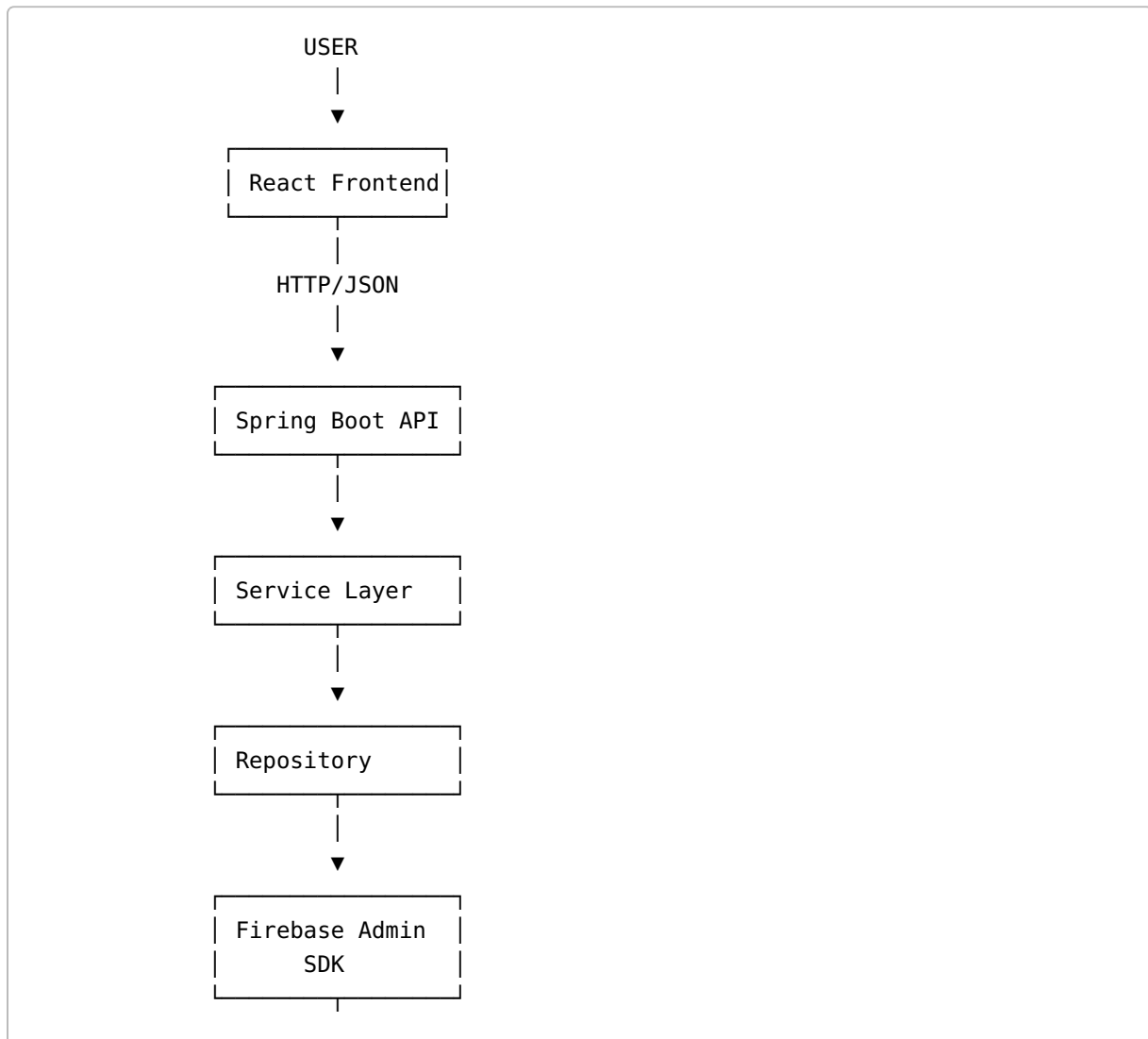
Prepare documentation according to the college SOP:

1. Title
2. Project Overview
3. Problem Statement
4. Objectives
5. Scope
6. Technology Stack
7. System Architecture
8. Firestore Database Design
9. UI Design
10. CRUD Implementation
11. REST API Documentation

- 12. Validation
- 13. Exception Handling
- 14. Testing
- 15. Test Results
- 16. Screenshots
- 17. Installation Procedure
- 18. Challenges and Solutions
- 19. Future Enhancements
- 20. GitHub Repository
- 21. Conclusion

47. ARCHITECTURE DIAGRAM

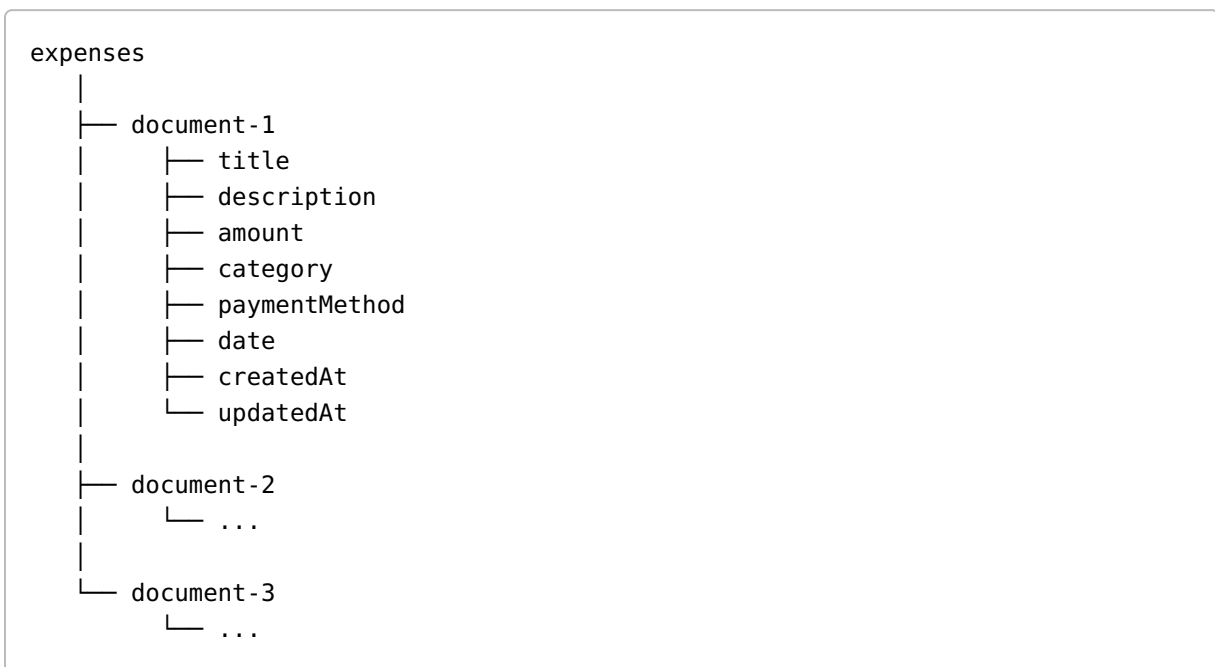
Include:





48. FIRESTORE STRUCTURE

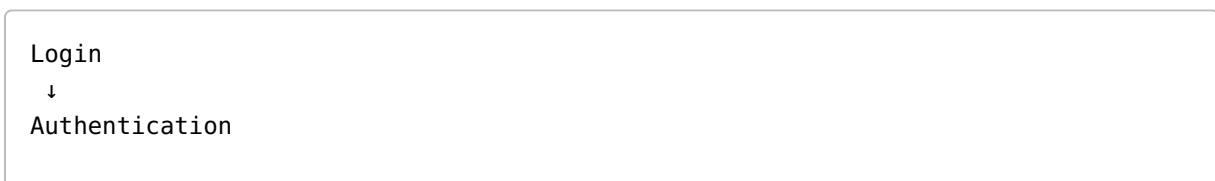
Document:



49. OPTIONAL AUTHENTICATION

Authentication can be added using Firebase Authentication.

If implemented:



↓
Dashboard

Provide:

- Login
- Logout
- Protected pages

However, authentication must not break the core CRUD functionality.

If not implemented, mention authentication as a future enhancement.

50. OPTIONAL BUDGET FEATURE

You may add a budget feature if it can be implemented cleanly.

Allow users to set:

Monthly Budget

Then display:

Monthly Budget	₹10,000
Spent	₹6,500
Remaining	₹3,500

Show a warning when spending approaches the budget.

Do not let optional features compromise the required CRUD functionality.

51. FUTURE ENHANCEMENTS

Document:

- Firebase Authentication
- User-specific expenses
- Monthly budgets
- Recurring expenses
- Income tracking

- Savings goals
 - Export CSV
 - Export PDF
 - Expense reminders
 - Advanced analytics
 - PWA/mobile application
 - Multi-user support
-

52. FINAL DEMONSTRATION

The application must be easy to demonstrate during viva.

Recommended demonstration sequence:

1. Explain architecture

```
React
↓
Spring Boot
↓
Firebase Firestore
```

2. Show Firebase

Open Firestore and show:

```
expenses
```

3. Show backend

Explain:

```
Expense.java
ExpenseRepository.java
ExpenseService.java
ExpenseController.java
```

4. Demonstrate Postman

Show:

```
POST
GET
GET/{id}
PUT
DELETE
```

5. Demonstrate frontend

Show:

```
Dashboard
↓
Add Expense
↓
Expense appears
↓
Search
↓
Filter
↓
Edit
↓
Delete
```

6. Demonstrate validation

Enter invalid amount or empty required fields.

7. Show Firebase

Refresh Firestore and demonstrate that the data has actually changed.

8. Show GitHub

Demonstrate:

- Repository
- Source code
- README
- Commit history

53. FINAL CRUD FLOW

CREATE:

```
React Form
↓
POST /api/expenses
↓
Spring Boot
↓
Firebase Firestore
↓
Success Response
↓
React
```

READ:

```
React
↓
GET /api/expenses
↓
Spring Boot
↓
Firestore
↓
JSON
↓
React
```

UPDATE:

```
React Edit Form
↓
PUT /api/expenses/{id}
↓
Spring Boot
↓
Firestore
↓
Updated data
```

↓
React

DELETE:

```
React
↓
DELETE /api/expenses/{id}
↓
Spring Boot
↓
Firestore
↓
Deleted
↓
React refresh
```

54. FINAL CHECKLIST

Before completion verify:

- ☐ React starts
- ☐ Spring Boot starts
- ☐ Firebase connection works
- ☐ Firestore works
- ☐ Add expense works
- ☐ View expenses works
- ☐ View individual expense works
- ☐ Edit expense works
- ☐ Delete expense works
- ☐ Search works
- ☐ Category filter works

- ☐ Payment method filter works
 - ☐ Date filter works
 - ☐ Sorting works
 - ☐ Dashboard statistics work
 - ☐ Charts use actual data
 - ☐ Validation works
 - ☐ Error handling works
 - ☐ Loading states work
 - ☐ Empty states work
 - ☐ Responsive UI works
 - ☐ Postman tests work
 - ☐ GitHub repository is clean
 - ☐ No secrets are committed
 - ☐ README is complete
 - ☐ Project report is complete
 - ☐ Screenshots are prepared
 - ☐ Architecture can be explained
 - ☐ CRUD flow can be demonstrated
-

55. IMPORTANT DEVELOPMENT RULE

Do not generate a fake frontend-only project.

Do not store the primary expense data only in React state.

Do not use localStorage as the primary database.

Do not use MySQL.

The actual database must be:

Firebase Firestore

The intended architecture is:

React → Spring Boot REST API → Firebase Firestore

Every CRUD operation must persist to Firestore.

56. DEVELOPMENT ORDER

Build incrementally in this exact order:

Milestone 1

Firebase project + Firestore

Milestone 2

Spring Boot project starts

Milestone 3

Firebase connection + Expense model

Milestone 4

POST + GET APIs

Milestone 5

PUT + DELETE APIs

Milestone 6

Postman testing

Milestone 7

React UI

Milestone 8

React ↔ Spring Boot ↔ Firebase integration

Milestone 9

Validation + search + filters + errors

Milestone 10

Dashboard + charts + responsive design

Milestone 11

Complete testing

Milestone 12

GitHub + README + screenshots

Milestone 13

Project report

Milestone 14

Final demonstration + viva preparation

57. FINAL EXPECTATION

Generate a complete working **Expense Tracker** full-stack web application.

The final application must be:

- Functional
- Responsive
- Professional
- Beginner-understandable

- College-project-ready
- CRUD compliant
- REST API compliant
- Firebase Firestore backed
- Properly validated
- Properly tested
- GitHub ready
- Documentation ready

Do not leave core features as TODOs or pseudo-code.

When credentials or Firebase configuration are required, use secure placeholders and provide exact setup instructions rather than exposing real credentials.